

VISUAL BATTLE INTEGRATION UPDATE

GENERAL

Purpose: define the visible timing and presentation requirements for battle processes. This section does not redefine DECAY rules, ownership, or architecture. Gameplay systems remain authoritative; presentation receives authoritative results and controls only how and when they are displayed.

DOC PRIORITIZATION : DECAY unity update/architecture doc → Visual Integration
Update→/CardHouse/Balatro code reference

The Visual Integration Update is authoritative about **what must eventually be communicated visually and when it belongs in the player-facing sequence**. It is not allowed to override ownership, editor-first presentation, state authority, or other implementation constraints established by the Unity Update.

(CardHouse, and Balatro are parallel reference sources. None are authoritative over the others; use whichever best clarifies the specific visual or implementation question, while the Visual Integration Update and DECAY rules/architecture remain authoritative.)

Rules determine what happened → Process Resolution Steps identify ordered processing beats → Facts identify authoritative changes → Presentation displays those processes and results → blocking presentation reports completion → flow continues.

Unless explicitly stated otherwise, gameplay outcomes, enemy decisions, and process results are determined authoritatively before their presentation begins; presentation may reveal or delay those results over time, but it does not determine or alter them.

dialogue will be added as its own later system later on, rather than embedding it into enemy reposition sequencing. Any mentions of dialogue are not prescriptive of code at this stage

DELAY: A short presentation-only pause inserted between visual beats to improve readability, rhythm, or emphasis. A Delay does not represent gameplay processing and does not alter authoritative state. Delays should be editor-configurable and may be shortened or skipped during simulation/instant presentation. A Delay may occur before, between, or after animations without requiring the preceding animation itself to be longer.

♠ to be implemented in battle phase pass

🏰 to be implemented in decay/score phase pass

♦ to be implemented in minor animations pass/general polish

♥ to be implemented in inventory pass later

♣ to be implemented at a later date

A symbol marks when the required infrastructure/behavior is implemented; authored art or final polish may carry a later symbol without delaying the underlying functionality.

Implementation should follow BALATRO and CARDHOUSE examples, leaving specifics and visuals as editor compatible and controlled rather than embedding things into scripts unnecessarily. If it CAN be manipulated easily and more fluidly in the editor, or if it needs to be later implemented by elise then prioritize editor.

SETUP PHASE

Entry condition: what triggers this phase

The phase present when the game starts by default. The phase following scoring. Dice are available at this point.

Authoritative information required: what must be present for the game to be playable

- visual representation of player dice within an inventory area
- Visual of enemy dice inside board slots
- Visual hourglass
- Visual round counter
- Visual Abacus

Visual Processes present: visuals that act over a period of time

- Idle animation for abacus ♦ for infrastructure ♣ for authored implement
- Idle animation of hand counter ♦ for infrastructure ♣ for authored implement
- Idle animation of enemy ♦ for infrastructure ♣ for authored implement
- Enemy populating board dice ♠

Eventual Visualizations: plans for final visuals. This can guide where animation/interaction architecture will need to go and should be easily integrated

- On Hover specific sprite for hourglass ♦
- Hover description box for enemy dice ♣
- drawer /inventory ♥
- Small animation for enemy board population ♦
- Dice settle on board small visual (code) ♦

- Dice swap animation ♦
- Dice drop and snap back to inv or board ♦ mostly ♥ later

Specific Needs for Visualizations: notes on how specific animations/sprites/visuals function in order to make integration clearer

- Hover and click should have different options for visualization ♦ ??
- Many Sprites use neutral state in this phase -round counter only uses neutral when a game starts or after round 4 is complete
- Dialogue may be an option later in this phase ♣

Process Notation: in a process, what visual states are gone through in what order after what condition

ENEMY BOARD POPULATION

1. Enemy setup is not a separate phase from player setup and should not restrict player availability
2. Enemy setup should be determined by enemy ai first, then be visualized
3. If player presses roll before enemy setup is visualized the enemy setup animation should be cut short and the roll animation activated immediately
4. If the enemy is not going to rearrange or change their board then no visuals will play
5. If enemy only changes some dice or not all, then visuals should be faster (aka, skip dice slots that do not change)
6. Dice should populate in order from slot 1 to 6 on the enemy board
7. Each dice population to a slot will show a small visual likely handled by editor/code. (see balatro juice for example) (see Balatro source)
8. Each dice Animation can overlap, and are very short. Enemy board population should not take more than 2 seconds

Editor configuration Needs

- Editor should have access to all small code animations for easing/curves. (See cardhouse and Balatro sources)
- Enemy board population should ideally have the ability to be slower or faster (see Balatro source) and this should be in editor
- Length of animation, speed, easing, sizing, etc for pops/wobbles.
- Places to integrate hover animation sprites in editor and decide hover state visuals as much as possible, including spacing, color, ease, pace

ROLL

Entry condition: what triggers this phase

The player clicks the Hourglass.

Authoritative information required: what must be present for the game to be playable

- Hourglass change state ♠
- Dice roll animation ♠
- Final faces shown after roll completes ♠
- Score values shown under dice slots on the board ♠
- Round counter changes to display round ♠

Visual Processes present: visuals that act over a period of time

- Hourglass flip
- Roll animation
- Round counter

Eventual Visualizations: plans for final visuals. This can guide where animation/interaction architecture will need to go and should be easily integrated

- Hourglass should flip over and as this happens it should change sprite to its visual sprite for the reposition phase
- Dice should all use the 2D roll animation

Specific Needs for Visualizations: notes on how specific animations/sprites/visuals function in order to make integration clearer

- The roll animation is authored 2D and should not be skipped, sped up , or slowed down unless simulation is happening.
- The roll animation will be different for different dice later in the game, but will follow the same timing
- When dice show their eventual face, there may be the necessity to add a neutral dice beneath the dice face and fade quickly into the final face. This is because the roll animation does not have specific faces (as this would mean every dice need 6 different roll animations), and so when it lands it displays a blank neutral face on its last frame.

Process Notation: in a process, what visual states are gone through in what order after what condition

ROLL

1. Hourglass is clicked
2. Immediately, roll animation for both sides starts at the same time the hourglass flip animation plays.
 - a. Roll animations may be offset slightly from each other by very minor amounts in order to make roll feel less mechanical. This amount will be randomized within a range.
 - b. final rolled faces are already authoritatively known before/during animation; animation does not determine them.
 - c. Enemy and player roll is simultaneous
3. After roll animation finishes, slight pop/fade animation into the faces on the dice.
 - a. During this the score values beneath the dice will populate, this is very quick but may have a slight anim eventually
4. Dice end roll animation process and enemy reposition phase begins

HOURGLASS

1. Hourglass is clicked
2. Begin (unknown if coded or authored) animation of flip
3. Final frame of flip animation finishes and the reposition phase hourglass sprite replaces the initial hourglass sprite.
4. The hourglass sprite after this is not clickable/hoverable until all dice have been rolled and show faces.

ROUND COUNTER

1. Hourglass is clicked
2. Small animation plays
3. Sprite changes from current to the sprite corresponding with next round.
4. **THE ROUND COUNTER ANIMATION ONLY OCCURS HERE IF THIS IS THE FIRST ROUND, OTHER WISE IT OCCURS AFTER SCORING**

Editor configuration Needs

- Editor should have place to add in the roll and flip animation, as well as their final sprite state
- Timing adjustments in editor would be best if possible

- For fade neutral face- treat as its own small animation after roll with its own initial sprite and the final sprite being the rolled face. Need a place for the initial sprite and this. It should take almost no time as it happens after roll.

ENEMY REPOSITION

Entry condition: roll animation over and face values have been determined.

Authoritative information required: what must be present for the game to be playable

- Visual of dice being moved/swapped ♠ ♦
- Option for enemy dialogue to be played and sprite to change ♣
- Indication of enemy reposition being over, likely a very slight visual cue ♠

Visual Processes present: visuals that act over a period of time

- **Enemy reposition animation (s)**

Eventual Visualizations: plans for final visuals. This can guide where animation/interaction architecture will need to go and should be easily integrated

- **Enemy may comment or react to rolled values, or on its AI decision. This means sprite/dialogue is dependant on both aspects and so should be able to react to both aspects**
- **A slight highlight of the enemy side of the board to indicate it is doing something and player cannot use controls yet**
- **Animations for repositioning (likely coded and similar to swap)**

Specific Needs for Visualizations: notes on how specific animations/sprites/visuals function in order to make integration clearer

- Enemy repositioning animation will be dependant on how many dice the enemy repositions/swaps.
- Even if no swaps occur, there should be a brief visual cue that enemy reposition has happened. ♠
- Swap animation can be the same as player dice swap animation.

Process Notation: in a process, what visual states are gone through in what order after what condition

ENEMY REPOSITION

1. Player side visually dims and/or enemy side visually highlights (very fast, almost immediate)
 - a. This always occurs even if no reposition will happen
2. Enemy does any reaction dialogue or sprite for roll
 - a. If there is none skip this beat
3. Enemy repositions dice
 - a. When the enemy repositions it will be swapping one dices position with another.
 - b. Each time a dices position is swapped it will play an animation that lasts a certain amount of time
4. Slight pause
 - a. This is always present, even if no enemy reposition occurred.
5. Enemy dialogue or sprite change for reposition. If none skip
6. Highlight fades and player reposition begins

Editor configuration Needs

- Editor should once again have a place to input enemy dialogue and sprites ♣
 - If there is no dialogue then do not populate a dialogue box. This is not tied to sprite.
 - Dialogue may be accompanied by a minor “talking” animation eventually -just to warn you
- Editor should be able to adjust intensity/ highlight properties, as it will likely be a unity only animation. Timing here as well
- Swap should have same control adjustment options as player swap.

REPOSITION

Entry condition: enemy reposition has ended

Authoritative information required: what must be present for the game to be playable

- Enemy dice faces shown
- Score values of dice

- Current score (will be addressed in score section)
- Dice faces on board
- Swap animation
- Hourglass state
- Round shown
- Targeted condition and idle ♠
 - predictive presentation state
- Decay idle ♠
 - predictive presentation states
- Save idle ♠
 - predictive presentation states

Visual Processes present: visuals that act over a period of time

- Swap
- targeted

Eventual Visualizations: plans for final visuals. This can guide where animation/interaction architecture will need to go and should be easily integrated

- Idle animations should resume during this time ♣ but can use ♦ and should have infrastructure
- Enemy dice hover descriptions should be available during this time ♣
- Inventory drawer should be openable but not interactable during this time
- Hover state when player is over enemy or player dice ♦
- Hover state for hourglass ♦
- Targeted state for a dice targeted by decay ♠
- Dice that will decay (either due to roll value, effects, or slot placement) show a idle animation ♠
 - This animation state is HIGHLY versatile and connected to the targeted condition
- Dice that can save (either due to roll value, effects, or slot placement) show an idle ♠
 - This is NOT repentant on if the save actually is going to be used. This idle will play even if a 1 is placed in slot 6 for example
 - Save and decay visuals should NOT be tied to the FACT or actual resolution. The player needs visual information BEFORE this in order to play and determine the resolution.

Specific Needs for Visualizations: notes on how specific animations/sprites/visuals function in order to make integration clearer

- Swap animation will be an in unity animation that should have options in editor
- Decay idle is authored
- Save idle is authored
- Targeted sprite is authored

Process Notation: in a process, what visual states are gone through in what order after what condition

SWAP

1. Dice clicked
2. Dice dragged to new position
3. Dice dropped into new position
4. Animation of dice in that slot moving to original dices slot ♦
5. Animation of dice settling ♦
 - a. This entire thing should be VERY quick and feel very good

TARGETED

TARGETED is a predictive presentation state derived from an authoritative Decay rule preview. It must not be stored as a competing mutable gameplay state or independently calculated by DiceView, SlotView, or other presentation code. During Reposition, presentation requests a read-only preview from the same authoritative rule logic used by the Decay process; this preview identifies which dice are currently TARGETED, which dice are causing that targeting, and any other information needed to display predictive Decay/Save/Effect indicators. The preview is refreshed after Roll and after any accepted reposition or other authoritative change that can affect the predicted Decay result. Future effects that alter targeting must extend the authoritative preview calculation rather than adding effect-specific targeting logic to presentation. TARGETED visuals end when Reposition ends and do not themselves create, cancel, or guarantee a later Decay result.

1. A condition that will cause decay occurs after roll process is completed (ie a six is rolled or an effect is triggered)
2. A dice that is going to be decayed is TARGETED.
 - a. A dice that is causing decay can also be targeted if another dice would decay it. This interaction is important for effects -ie a dice rolled a 6 and is causing the dice opposite it to decay, but it has an effect that means the first time it would decay it does not. HOWEVER then opposing side has an 6 roll dice with an effect that triggers decay twice on any spot. So it is

targeted. This is IMPORTANT because it makes it clear WHEN decay will occur when effects become layered or complex.

3. A targeted dice shows a sprite around it that visualizes the targeted state
 - a. This sprite will likely have a very tiny and fast start/stop anim
 - b. This sprite is going to have aspects that are ALPHA 0 or TRANSPARENT, and will be positioned on top of existing dice. This is so the targeting sprite/animation does not need to be redone every time a new dice type is added to the game
 - c. A targeted dice can be playing the decay idle while targeted
 - d. A targeted dice can play a save idle while targeted
 - e. A targeted dice can play any necessary effect idles while targeted.
4. When a dice is untargeted the sprite disappears, when a dice is targeted the sprite appears
5. When the reposition phase is over all targeted animations stop.

All effect anime are ♣ **but the architecture must be in during ♠ to allow for them**

Editor configuration Needs

- Editor should have place to adjust curves etc. check balatro and carhouse for examples
- Places to input hover state sprites or anims
- Targeted intro/exit options ♦
- Effect idles sprite/animation placement/editor options
- Targeted sprite option
- save/decay idle sprite/anim options

DECAY (THE BIG ONE)

Entry condition: the player clicks the hourglass after the roll/reposition phase

Authoritative information required: what must be present for the game to be playable

- Decay has occurred on this dice animation
- Save has occurred on this dice animation/ this dice has saved animation
- This dice has been checked animation
- Infrastructure for effect animations occurring
- Slot breaks animation
- Slot is unstable animation

Visual Processes present: visuals that act over a period of time

- **Decay process**
 - **Save** 🎲
 - **Decay** 🎲
 - **Slot** 🎲
 - **Effect** 🎲 ♣️ **authored implemented afterwards**
 - **Checked** 🎲

Eventual Visualizations: plans for final visuals. This can guide where animation/interaction architecture will need to go and should be easily integrated

- **Decay and save will have authored animations**
- **Many effects will have authored animations that function similarly to decay/save**
- **Slot animations are tied to decay or effects and occur sequentially after these**

Process Notation: in a process, what visual states are gone through in what order after what condition

PROCESS STAGGER AND OVERLAP: Each eligible SlotPair begins its visualization after a fixed, editor-configurable start-to-start stagger from the previous eligible SlotPair. The next SlotPair does not wait for the previous SlotPair's animations or effects to finish. Animations may overlap and may finish out of order if one effect is longer than another. Broken/ineligible SlotPairs consume no visual beat or stagger and are skipped immediately. After SlotPair 6 has begun, the process does not advance to the next phase until all remaining animations belonging to the process have completed.

DECAY

Decay visualization should use the authoritative ordered pair-resolution output rather than reconstructing Decay from final board state or Facts alone. **DecayPairResolution** represents one opposing numbered pair in the Decay process and preserves the established simultaneous $1E/1P \rightarrow 2E/2P \rightarrow \dots \rightarrow 6E/6P$ structure. It may be expanded to expose the authoritative information presentation requires for that pair—such as participating dice, eligibility, Decay/Save outcomes, sources and targets, and resulting slot conditions—without becoming mutable gameplay state or containing animation timing. Presentation uses these ordered results to determine which visual beat and outcome animations to show; Facts continue to describe actual authoritative state changes. Visual timing, overlap, and animation choice remain

presentation responsibilities.

- 1. The hourglass is clicked**
- 2. The first slot begins its visualization**
 - a. This means the enemy and player slot 1 animate simultaneously**
 - b. If the slot has no decay/save/effect a animation is played to show the slot has been processed**
 - c. If the slot is broken then no animation occurs and the timing for that slot is skipped**
 - d. If something in this slot decays then that animation plays**
 - i. The decay animation is overlaid ontop of the dice it effects. Do not remove the visible face sprite for the dice until the animation for decay has completed**
 - 1. This is true of save/most effects because it makes animations compatible with all dice types. (aka save can effect dice of any shape/size/color without having to be redone)**
 - ii. After decay animation has happened, the slot break animation plays (unless an effect negates this in which case the timing is skipped)**
 - e. If something in this slot is saved then that animation plays**
 - i. When a save occurs, the saved and the saving dice both play the animation simultaneously**
 - ii. After a save the unstable animation happens**
 - f. If an effect occurs that animation is played (unless an effect negates this in which case the timing is skipped)**
- 3. The second slot begins its processing after the first slot has begun its visualization and after a certain amount of time has passed (a minor pause, look at balatro source for ref) (PROCESS STAGGER AND OVERLAP)**
 - a. This means slot 1 and 2 visuals can overlap**
- 4. This visual process continues with the same amount of time waited between each slot, going from 1 to 6**
 - a. Once again enemy and player opposing slots show visuals simultaneously**
 - b. If a slot is broken then its timing is skipped**
 - c. Effects in this time should only be decay/save relevant effects. All score effects will be processed during the score phase.**
 - d. If a dice has multiple effects/conditions happening (unlikely but you never know) then play decay last always and save first. Slot animations should always follow the final effect/save/decay**

animation. This only applies to explicitly mean visual ordering within one already-resolved pair, not gameplay/effect resolution priority.

5. After all non-broken slots have had visualization finish decay is over and score begins

Editor configuration Needs

- No idea

SCORE

Entry condition: decay process is over

Authoritative information required: what must be present for the game to be playable

- Each dice shows a tiny animation 🎲/♦
- Abacus score updates 🏠

Visual Processes present: visuals that act over a period of time

- **SCORE PROCESS**

Eventual Visualizations: plans for final visuals. This can guide where animation/interaction architecture will need to go and should be easily integrated

- **Aspects of this will be authored (abacus reaction in sprite and any effect triggers)**
- **Generally faster than decay**

Specific Needs for Visualizations: notes on how specific animations/sprites/visuals function in order to make integration clearer

- Both player and enemy again simultaneous
- Abacus score numerals should have a visual aspect

Process Notation: in a process, what visual states are gone through in what order after what condition

SCORE

1. Slight pause after decay finishes
2. Dice in first unbroken slot **-every eligible/non-broken slot should get a processing beat even if it contributes no score; broken slots create no pause.** - does a very slight bounce/hover/tilt (undecided currently) AND its displayed score value pulses then disappears.
 - a. **This is VERY fast**
 - b. **Score pulse, abacus addition, and animation should all be at the same time** 🏠/♦
 - c. **The position after this animation should be a separate held sprite called SCORE or something similar (more on this later, only presentation state)**
 - d. **If a dice has a SCORING effect, this is played here** ♣
 - e. **score value disappearing is tied to the final animation played**
 - f. **After the first dice begins to score the abacus begins its score animation state, and will hold this state until all slots have scored visually**
3. After a designated amount of time (normally the time for the default animation without an effect) the next unbroken slot in order does the score animations **(PROCESS STAGGER AND OVERLAP)**
 - a. This continues for all unbroken slots INCLUDING unstable.
 - i. For unstable slots, only one side will have the animation play as the other is broken. This slot should NOT be skipped however, as it does score
4. After all slots have been scored and finished their visual scoring the abacus does its FINAL score animation
 - a. A very short authored animation accompanied by a score board finalization
 - b. **AT THE SAME TIME** this animation plays, all dice that have been held in the SCORE sprite play a final reset animation simultaneously where they pulse (or something similar) and then reset to neutral

RESET

Entry condition: scoring ends

Authoritative information required: what must be present for the game to be playable

- Neutral dice state 🏠
- Round counter progression 🏠
- Abacus visuals stay 🏠

- Unstable resolution 🏰
- Hourglass reset 🕒

Visual Processes present: visuals that act over a period of time

- **unstable**
- **Hourglass**
- **Round counter**

Eventual Visualizations: plans for final visuals. This can guide where animation/interaction architecture will need to go and should be easily integrated

- **Hourglass animation similar to others where it flips back to its setup phase state**
- **Unstable slot will transition to a broken slot here**
- **Round counter will move up 1 UNLESS game is over in which case it resets to neutral**

Process Notation: in a process, what visual states are gone through in what order after what condition

HOURLASS

1. Score is over
2. Flip animation plays
3. Setup phase sprite displayed and active

UNSTABLE

1. Score is over
2. ALL unstable slots play animation (coded similar to dice snap back animation) of dice being put back into inventory
 - a. If this happens to enemy dice then dice move towards sprite and fade rather than inventory
 - b. Inventory does not need to open or respond, and this visual is very quick but not a snap -slightly more time and weight. Will be best to have editor options here
3. After dice return animation finishes, a slight stamp/fade plays on all unstable slots changing them to broken

ROUND COUNTER

1. Score ends

2. Round counter animation plays
3. Next round sprite shown
 - a. If it is final round play to neutral